ECE

Major: Cybersecurity International

Course
Sécurité des Containers

Lab Report
Projet Final : Minecraft Secure Cluster

Instructor
Etienne LOUTSCH

Group Members

- ARDILLON Bastian
- SAVEAUX Octave
- BRULEY Martin

Date
Mars 2026

Sommaire

1. Étapes du projet
2. Capture d'écran du serveur Minecraft accessible
3. Documentation des choix de sécurité
4. Exemple de rejet de pipeline pour CVE critique
5. Bonus

Étapes du projet

Nous avons utilisé une machine Ubuntu 22.04 LTS avec Docker, kubectl, Kind et Helm installés. Le cluster Kind a été configuré avec un fichier `cluster.yml` incluant un port mapping pour exposer le serveur Minecraft :

Le port mapping redirige le port 25565 (port par défaut de Minecraft) de la machine hôte vers le port 30001 du cluster

Après la création du cluster, nous avons vérifié que le nœud est bien en état Ready :

```
kubectl get nodes
```

```
root@PCoctave:~/minecraft-secure-cluster# kind create cluster --name minecraft-cluster --config cluster.yml
Creating cluster "minecraft-cluster" ...
 ✓ Ensuring node image (kindest/node:v1.27.3) 🖼
 ✓ Preparing nodes 📦
 ✓ Writing configuration 📜
 ✓ Starting control-plane 🕹️
 ✓ Installing CNI 🔌
 ✓ Installing StorageClass 💾
Set kubectl context to "kind-minecraft-cluster"
You can now use your cluster with:

kubectl cluster-info --context kind-minecraft-cluster

Not sure what to do next? 😅  Check out https://kind.sigs.k8s.io/docs/user/quick-start/
root@PCoctave:~/minecraft-secure-cluster# kubectl get nodes
NAME                              STATUS   ROLES           AGE   VERSION
minecraft-cluster-control-plane   Ready    control-plane   54s   v1.27.3
root@PCoctave:~/minecraft-secure-cluster#
```

Figure 1 : nœud du cluster Kind en état Ready

Pour pouvoir déployer depuis GitLab, nous avons installé un GitLab Runner directement dans notre cluster. Nous avons directement suivi les instructions données dans le sujet du projet.

Le Runner apparaît bien comme connecté dans l'interface GitLab :

GitLab Runner connecté

Figure 2 : GitLab Runner connecté dans l'interface GitLab

Puis nous avons vérifié que le pod du Runner est en état Running :

```
NAME: gitlab-runner
LAST DEPLOYED: Sun Mar 15 14:37:06 2026
NAMESPACE: gitlab-runner
STATUS: deployed
REVISION: 1
DESCRIPTION: Install complete
TEST SUITE: None
NOTES:
Your GitLab Runner should now be registered against the GitLab instance reachable at: "https://gitlab.com"

Runner namespace "gitlab-runner" was found in runners.config template.
root@PCoctave:~/minecraft-secure-cluster# kubectl get pods -n gitlab-runner
NAME                              READY   STATUS    RESTARTS   AGE
gitlab-runner-98484c98d-wbg2g     0/1     Running   0          34s
root@PCoctave:~/minecraft-secure-cluster#
```

Figure 3 : pods du GitLab Runner en état Running

Nous avons par la suite créé un namespace `minecraft` , puis configuré un ServiceAccount, un Role et un RoleBinding pour que la CI puisse déployer dans ce namespace :

Nous avons ensuite généré un token pour le ServiceAccount et l'avons stocké comme variable CI/CD dans GitLab :

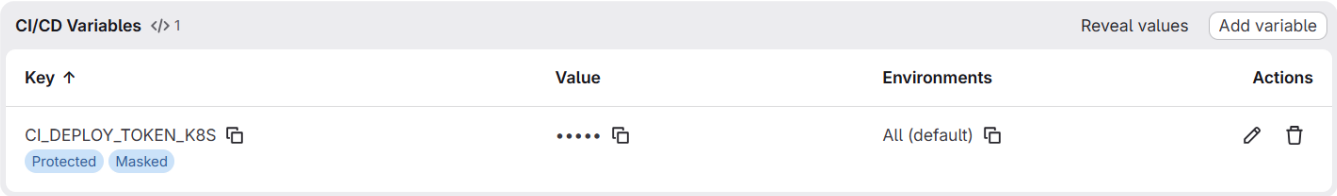

Figure 4 : variable CI_DEPLOY_TOKEN_K8S ajoutée dans GitLab (Protected & Masked)

Le Dockerfile est basé sur l'image officielle `itzg/minecraft-server` :

Le fichier `minecraft-deploy.yml` contient un Deployment et un Service. Le Deployment configure le serveur avec les variables d'environnement nécessaires. Le Service de type NodePort expose le port 25565 sur le nodePort 30001 :

Nous avons vérifié que le pod Minecraft démarre correctement :

```
root@PCoctave:~/minecraft-secure-cluster# kubectl get pods -n minecraft
NAME                          READY   STATUS    RESTARTS        AGE
minecraft-7fcd6f5594-ml6q9    1/1     Running   1 (2m47s ago)   3m9s
root@PCoctave:~/minecraft-secure-cluster#
```

Figure 5 : pod Minecraft en état Running

Les logs du serveur confirment le démarrage avec le message "Done" :

```
[13:56:27] [ServerMain/INFO]: Environment: Environment[sessionHost=https://sessionserver.mojang.com, servicesHost=https://api.m
inecraftservices.com, profilesHost=https://api.mojang.com, name=PROD]
[13:56:29] [ServerMain/INFO]: No existing world data, creating new world
[13:56:29] [ServerMain/WARN]: Failed to parse level-type default, defaulting to minecraft:normal
[13:56:31] [ServerMain/INFO]: Loaded 1470 recipes
[13:56:31] [ServerMain/INFO]: Loaded 1584 advancements
[13:56:31] [Server thread/INFO]: Starting minecraft server version 1.21.11
[13:56:31] [Server thread/INFO]: Loading properties
[13:56:31] [Server thread/INFO]: Default game type: SURVIVAL
[13:56:31] [Server thread/INFO]: Generating keypair
[13:56:31] [Server thread/INFO]: Starting Minecraft server on *:25565
[13:56:32] [Server thread/INFO]: Preparing level "world"
[13:56:33] [Server thread/INFO]: Selecting global world spawn...
[13:56:50] [Server thread/INFO]: Loading 0 persistent chunks...
[13:56:50] [Server thread/INFO]: Preparing spawn area: 100%
[13:56:50] [Server thread/INFO]: Time elapsed: 14804 ms
[13:56:50] [Server thread/INFO]: Done (15.793s)! For help, type "help"
```

Figure 6 : message "Done" confirmant le démarrage du serveur Minecraft

---

Le pipeline `.gitlab-ci.yml` est composé de 5 étapes :

- `lint` : analyse du code
- `build` : construit l'image Docker

- `scan` : détecte les CVE
- `verify` : analyse les CVE HIGH/MEDIUM
- `deploy` : déploie le serveur dans Kubernetes

La pipeline s'exécute avec succès, sauf le stage `scan` qui est en warning car des CVE critiques ont été détectées :

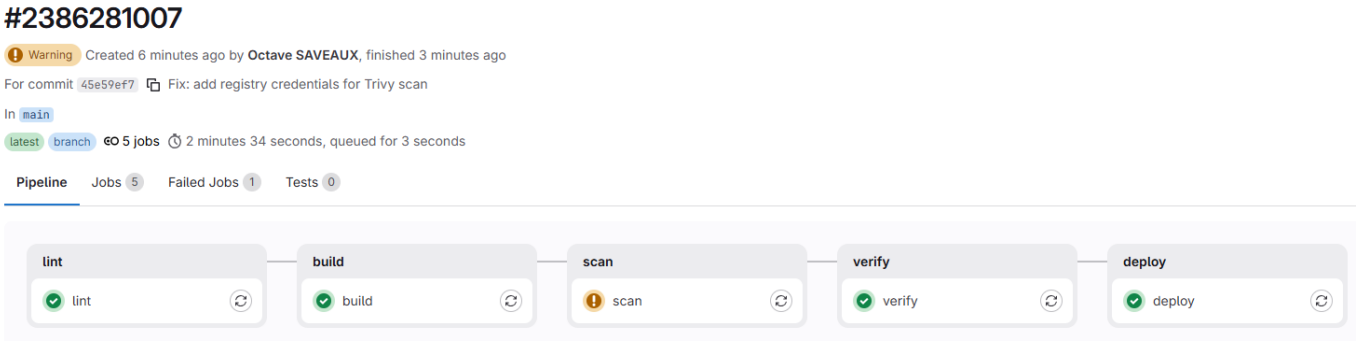

Figure 7 : pipeline CI/CD — tous les stages passent sauf le scan (warning attendu)

Après le déploiement par la CI/CD, le pod Minecraft est toujours en état Running :

```
root@PCoctave:~/minecraft-secure-cluster# kubectl get pods -n minecraft
NAME                          READY   STATUS    RESTARTS      AGE
minecraft-7fcd6f5594-ml6q9    1/1     Running   1 (49m ago)   49m
root@PCoctave:~/minecraft-secure-cluster#
```

Figure 8 : pod Minecraft en Running après déploiement via la CI/CD

Capture d'écran du serveur Minecraft accessible

Le serveur Minecraft est accessible en se connectant à l'adresse `127.0.0.1:25565`.

Ajout du serveur dans Minecraft :

Ajout du serveur Minecraft

Figure 9 : ajout du serveur avec l'adresse 127.0.0.1:25565

Le serveur apparaît dans la liste multijoueur avecle message "ECE 2026 - Server!" :

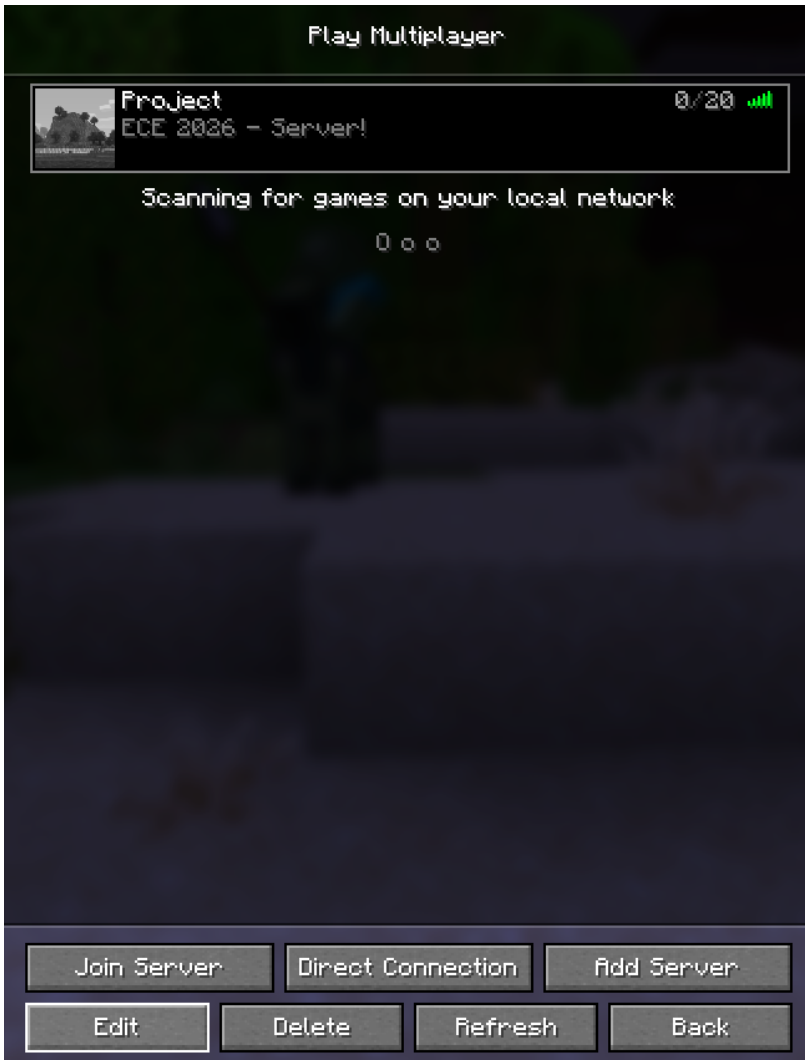

Figure 10 : serveur visible dans la liste multijoueur

Connexion réussie au serveur :

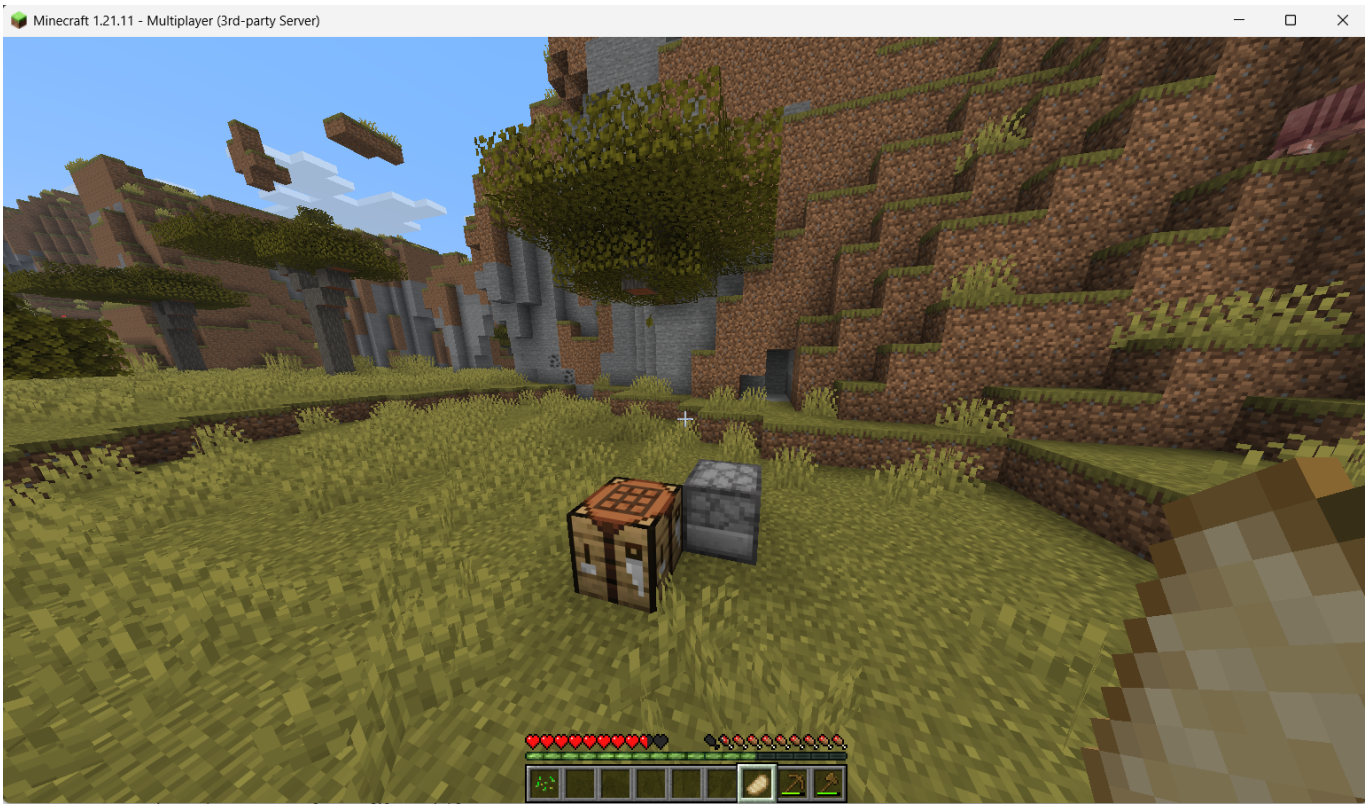

Figure 11 : connexion réussie au serveur Minecraft — le joueur est dans le monde

---

Documentation des choix de sécurité

Choix de sécurité implémentés

1. RBAC (Role-Based Access Control) Le ServiceAccount `ci-deployer` dispose uniquement des permissions nécessaires au déploiement dans le namespace `minecraft`. Il ne peut pas accéder aux autres namespaces ni effectuer d'opérations en dehors de son périmètre. Ce principe du **moindre privilège** limite l'impact en cas de compromission du token CI.

2. Kaniko pour le build d'images Nous avons choisi Kaniko pour construire les images. Kaniko ne nécessite pas de privilèges root, il peut donc s'exécuter entièrement en espace utilisateur dans le cluster Kubernetes.

3. Trivy pour le scan de vulnérabilités Trivy analyse automatiquement l'image construite à chaque pipeline. Le stage `scan` est configuré avec `--exit-code 1 --severity CRITICAL` pour bloquer le déploiement si des CVE critiques sont détectées. Le stage `verify` analyse les vulnérabilités HIGH/MEDIUM à titre informatif.

4. Variables protégées et masquées Le token `CI_DEPLOY_TOKEN_K8S` est stocké en tant que variable CI/CD **Protected** (utilisable uniquement sur les branches protégées) et **Masked** (non visible dans les logs de la pipeline).

5. Network Policies (Bonus) Une NetworkPolicy restreint le trafic entrant vers le pod Minecraft au seul port 25565. Tout autre trafic réseau entrant est bloqué, réduisant la surface d'attaque.

6. Falco Monitoring (Bonus) Falco surveille en temps réel les activités suspectes sur le cluster. Il détecte notamment l'ouverture de shells dans les containers et la lecture de fichiers sensibles, ce qui permet de détecter des tentatives de triche ou d'intrusion sur le serveur Minecraft.

Exemple de rejet de pipeline pour CVE critique

Lorsque des CVE critiques sont détectées dans l'image, le job échoue. Grâce au paramètre `allow_failure: true`, la pipeline continue mais le stage apparaît en **warning**.

Trivy a détecté les CVE critiques suivantes dans l'image `itzg/minecraft-server` :

Bibliothèque	CVE	Sévérité	Version installée	Version corrigée
scala-library	CVE-2022-36944	CRITICAL	2.13.1	2.13.9
easy-add (Go stdlib)	CVE-2025-68121	CRITICAL	v1.24.4	1.24.13, 1.25.7
gosu (Go stdlib)	CVE-2025-68121	CRITICAL	v1.24.6	1.24.13, 1.25.7

- **CVE-2022-36944** : vulnérabilité dans la bibliothèque Scala, permettant l'exécution de code.
- **CVE-2025-68121** : problème de reprise de session inattendue dans `crypto/tls` de Go, pouvant compromettre la sécurité TLS.

```
2026-03-15T14:36:32.271542Z 010
2026-03-15T14:36:32.271548Z 010 | Library | Vulnerability | Severity | Status | Installed Version | Fixed Version | Title |
2026-03-15T14:36:32.271578Z 010
2026-03-15T14:36:32.271597Z 010 | org.scala-lang:scala-library (scala-library-2.13.1.jar) | CVE-2022-36944 | CRITICAL | fixed | 2.13.1 | 2.13.9 | scala: deserialization gadget chain https://avd.aquasec.com/nvd/cve-2022-36944 |
2026-03-15T14:36:32.271600Z 010
2026-03-15T14:36:32.271603Z 010
2026-03-15T14:36:32.271606Z 010
2026-03-15T14:36:32.271608Z 010 usr/bin/easy-add (gobinary)
2026-03-15T14:36:32.271610Z 010 ==========================
2026-03-15T14:36:32.271611Z 010 Total: 1 (CRITICAL: 1)
2026-03-15T14:36:32.271613Z 010
2026-03-15T14:36:32.271618Z 010
2026-03-15T14:36:32.271621Z 010 | Library | Vulnerability | Severity | Status | Installed Version | Fixed Version | Title |
2026-03-15T14:36:32.271623Z 010
2026-03-15T14:36:32.271627Z 010 | stdlib | CVE-2025-68121 | CRITICAL | fixed | v1.24.4 | 1.24.13, 1.25.7, 1.26.0-rc.3 | crypto/tls: Unexpected session resumption in crypto/tls https://avd.aquasec.com/nvd/cve-2025-68121 |
2026-03-15T14:36:32.271628Z 010
2026-03-15T14:36:32.271639Z 010
2026-03-15T14:36:32.271643Z 010
2026-03-15T14:36:32.271644Z 010 usr/local/bin/gosu (gobinary)
2026-03-15T14:36:32.271646Z 010 ============================
2026-03-15T14:36:32.271648Z 010 Total: 1 (CRITICAL: 1)
2026-03-15T14:36:32.271652Z 010
2026-03-15T14:36:32.271655Z 010
2026-03-15T14:36:32.271658Z 010 | Library | Vulnerability | Severity | Status | Installed Version | Fixed Version | Title |
2026-03-15T14:36:32.271661Z 010
2026-03-15T14:36:32.271664Z 010 | stdlib | CVE-2025-68121 | CRITICAL | fixed | v1.24.6 | 1.24.13, 1.25.7, 1.26.0-rc.3 | crypto/tls: Unexpected session resumption in crypto/tls https://avd.aquasec.com/nvd/cve-2025-68121 |
2026-03-15T14:36:32.271667Z 010
2026-03-15T14:36:32.271669Z 010
2026-03-15T14:36:32.271690Z 010
```

Figure 12 : CVE critiques détectées par Trivy lors du scan de l'image

Ces vulnérabilités proviennent de l'image de base `itzg/minecraft-server` et de ses dépendances. Il faut donc mettre a jour les dépendances concernées aux versions recommandées.

Bonus

5.1 Network Policies

Nous avons créé une NetworkPolicy pour restreindre l'accès réseau au pod Minecraft. Seul le trafic entrant sur le port 25565 (TCP) est autorisé :

```yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: minecraft-network-policy
  namespace: minecraft
spec:
  podSelector:
    matchLabels:
      app: minecraft
  policyTypes:
    - Ingress
  ingress:
    - ports:
        - protocol: TCP
          port: 25565
```

```
  GNU nano 6.2                                          network-policy.yml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: minecraft-network-policy
  namespace: minecraft
spec:
  podSelector:
    matchLabels:
      app: minecraft
  policyTypes:
    - Ingress
  ingress:
    - ports:
        - protocol: TCP
          port: 25565
```

Figure 13 : contenu du fichier network-policy.yml

La policy a été appliquée avec succès :

```
root@PCoctave:~/minecraft-secure-cluster# kubectl get networkpolicy -n minecraft
NAME                        POD-SELECTOR    AGE
minecraft-network-policy    app=minecraft   20s
root@PCoctave:~/minecraft-secure-cluster#
```

Figure 14 : NetworkPolicy minecraft-network-policy créée

Nous avons vérifié que le serveur Minecraft reste accessible depuis le jeu après l'application de la policy :

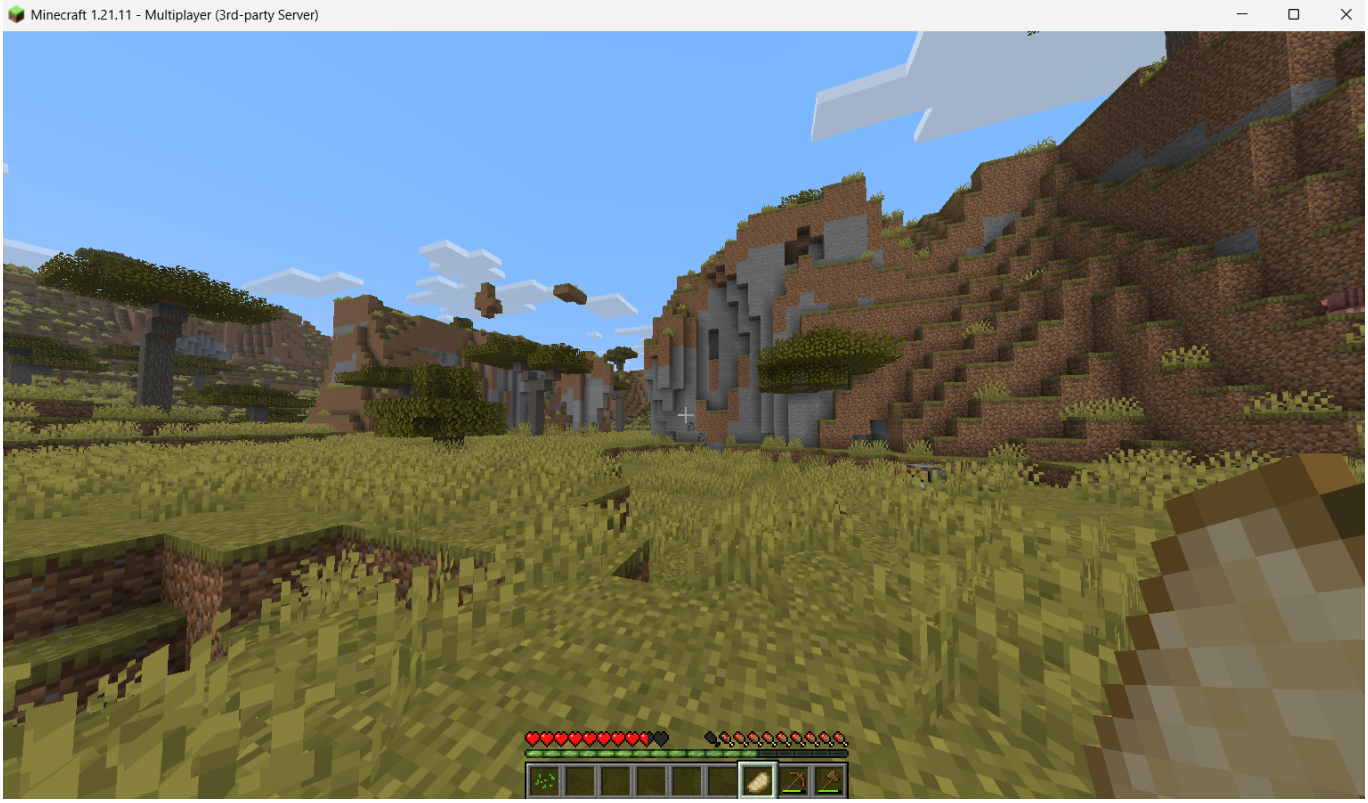

Figure 15 : serveur Minecraft toujours accessible sur le port 25565

Pour prouver que la policy fonctionne, nous avons lancé un pod de test dans le namespace `minecraft` et tenté d'accéder au service sur un port non autorisé. La connexion est bien bloquée:

Port 80 bloqué par la NetworkPolicy

Figure 16 : port 25565 ouvert vs port 80 bloqué

5.2 Falco Monitoring

Nous avons installé Falco dans le cluster pour surveiller les activités suspectes sur les containers, notamment le pod Minecraft

Nous avons vérifié que les pods Falco sont en état Running :

```
root@PCoctave:~/minecraft-secure-cluster# kubectl get pods -n falco
NAME                                      READY   STATUS    RESTARTS   AGE
falco-falcosidekick-5f4bbb544b-7g4r9      1/1     Running   0          14m
falco-falcosidekick-5f4bbb544b-8vrpv      1/1     Running   0          14m
falco-falcosidekick-ui-5c97b4c869-47djv   1/1     Running   0          14m
falco-falcosidekick-ui-5c97b4c869-w7r6w   1/1     Running   0          14m
falco-falcosidekick-ui-redis-0            1/1     Running   0          14m
falco-k8cjs                               2/2     Running   0          14m
falco-k8s-metacollector-845595f65d-85dn8  1/1     Running   0          14m
root@PCoctave:~/minecraft-secure-cluster#
```

Figure 17 : pods Falco en état Running

Pour simuler une tentative de triche nous avons exécuté un shell dans le pod et effectué les commandes suivante :

```
whoami
cat /etc/shadow
exit
```

```
root@minecraft-7fcd6f5594-ml6q9:/data# whoami
root
root@minecraft-7fcd6f5594-ml6q9:/data# ls -la /root/
total 24
drwx------ 1 root root 4096 Mar 16 16:30 .
drwxr-xr-x 1 root root 4096 Mar 16 16:28 ..
-rw------- 1 root root   28 Mar 16 16:30 .bash_history
-rw-r--r-- 1 root root 3106 Apr 22  2024 .bashrc
-rw-r--r-- 1 root root  161 Apr 22  2024 .profile
-rw-r--r-- 1 root root  165 Feb 17 20:20 .wget-hsts
root@minecraft-7fcd6f5594-ml6q9:/data# cat /etc/shadow
root:*:20494:0:99999:7:::
```

Figure 18 : exécution de commandes suspectes dans le container Minecraft

Falco a immédiatement détecté ces activités et généré des alertes :


```
{"hostname":"minecraft-cluster-control-plane","output":"16:34:25.118370330: Notice A shell was spawned in a container with an a
ttached terminal | evt_type=execve user=root user_uid=0 user_loginuid=-1 process=bash proc_exepath=/usr/bin/bash parent=runc co
mmand=bash -c cat /etc/shadow; whoami; ls -la /root terminal=34816 exe_flags=EXE_WRITABLE|EXE_LOWER_LAYER k8smeta_pod_name=mine
craft-7fcd6f5594-ml6q9 k8smeta_ns_name=minecraft container_id=62a2e3797c29 container_name=minecraft container_image_repository=
docker.io/itzg/minecraft-server container_image_tag=latest k8s_pod_name=minecraft-7fcd6f5594-ml6q9 k8s_ns_name=minecraft","outp
ut_fields":{"container.id":"62a2e3797c29","container.image.repository":"docker.io/itzg/minecraft-server","container.image.tag":
"latest","container.name":"minecraft","evt.arg.flags":"EXE_WRITABLE|EXE_LOWER_LAYER","evt.time":1773678865118370330,"evt.type":
"execve","k8s.ns.name":"minecraft","k8s.pod.name":"minecraft-7fcd6f5594-ml6q9","k8smeta.ns.name":"minecraft","k8smeta.pod.name"
:"minecraft-7fcd6f5594-ml6q9","proc.cmdline":"bash -c cat /etc/shadow; whoami; ls -la /root","proc.exepath":"/usr/bin/bash","pr
oc.name":"bash","proc.pname":"runc","proc.tty":34816,"user.loginuid":-1,"user.name":"root","user.uid":0},"priority":"Notice","r
ule":"Terminal shell in container","source":"syscall","tags":["T1059","container","maturity_stable","mitre_execution","shell"],
"time":"2026-03-16T16:34:25.118370330Z"}
{"hostname":"minecraft-cluster-control-plane","output":"16:34:25.120495756: Warning Sensitive file opened for reading by non-tr
usted program | file=/etc/shadow gparent=<NA> ggparent=<NA> gggparent=<NA> evt_type=openat user=root user_uid=0 user_loginuid=-
1 process=cat proc_exepath=/usr/bin/cat parent=bash command=cat /etc/shadow terminal=34816 k8smeta_pod_name=minecraft-7fcd6f559
4-ml6q9 k8smeta_ns_name=minecraft container_id=62a2e3797c29 container_name=minecraft container_image_repository=docker.io/itzg/
minecraft-server container_image_tag=latest k8s_pod_name=minecraft-7fcd6f5594-ml6q9 k8s_ns_name=minecraft","output_fields":{"co
ntainer.id":"62a2e3797c29","container.image.repository":"docker.io/itzg/minecraft-server","container.image.tag":"latest","conta
iner.name":"minecraft","evt.time":1773678865120495756,"evt.type":"openat","fd.name":"/etc/shadow","k8s.ns.name":"minecraft","k8
s.pod.name":"minecraft-7fcd6f5594-ml6q9","k8smeta.ns.name":"minecraft","k8smeta.pod.name":"minecraft-7fcd6f5594-ml6q9","proc.an
ame[2]":null,"proc.aname[3]":null,"proc.aname[4]":null,"proc.cmdline":"cat /etc/shadow","proc.exepath":"/usr/bin/cat","proc.nam
e":"cat","proc.pname":"bash","proc.tty":34816,"user.loginuid":-1,"user.name":"root","user.uid":0},"priority":"Warning","rule":"
Read sensitive file untrusted","source":"syscall","tags":["T1555","container","filesystem","host","maturity_stable","mitre_cred
ential_access"],"time":"2026-03-16T16:34:25.120495756Z"}
```


Figure 19 : alertes Falco

```
2026-03-16T16:30:01.895966023Z | Terminal shell in container | bash
2026-03-16T16:32:32.986241774Z | Terminal shell in container | bash -c cat /etc/passwd && whoami
2026-03-16T16:34:25.118370330Z | Terminal shell in container | bash -c cat /etc/shadow; whoami; ls -la /root
2026-03-16T16:34:25.120495756Z | Read sensitive file untrusted | cat /etc/shadow
2026-03-16T16:42:07.770212024Z | Terminal shell in container | bash
2026-03-16T16:42:36.101919308Z | Read sensitive file untrusted | cat /etc/shadow
2026-03-16T16:48:14.613979901Z | Terminal shell in container | bash
2026-03-16T16:49:05.574007152Z | Terminal shell in container | bash -c whoami && cat /etc/shadow
root@PCoctave:~/minecraft-secure-cluster#
```

Figure 20 : alertes Falco filtrées"

Falco a détecté deux types d'alertes :

- **Terminal shell in container** : détection de l'ouverture d'un shell bash dans le container Minecraft, correspondant à une potentielle tentative d'accès non autorisé.

- **Read sensitive file untrusted** : détection de la lecture du fichier `/etc/shadow`, indiquant une tentative d'accès à des informations sensibles du système.